Informe de testing y cobertura

Master Martini | DSY1104 EP2 | 20 octubre 2026

La ejecución final de Jasmine y Karma en ChromeHeadless aprobó 71 especificaciones, sin fallos. Se verificaron componentes, eventos, formularios, navegación, persistencia, CRUD y checkout. Este informe distingue cobertura frontend, pruebas complementarias Node y comprobaciones de navegador; no atribuye métricas de Node a React.

Entorno y ejecución

Windows, Node.js 24.15.0, React 18, Karma 6.4.4 y Chrome Headless 155. Babel transforma JSX e instrumenta con babel-plugin-istanbul; Webpack construye el paquete de pruebas. karma-coverage genera HTML, LCOV, JSON completo y resumen JSON. npm install finalizó correctamente y package-lock.json fija la instalación.

Comprobación	Resultado	Evidencia en testing/evidence/
Sintaxis y lógica	Aprobadas	check-syntax.txt, check-logic.txt
Node lógica y persistencia	39 aprobadas, 0 fallidas	test-node.txt
Assets locales	4 aprobadas, 0 fallidas	test-assets.txt
API HTTP local	15 aprobadas, 0 fallidas	test-api.txt
Build producción Vite	Aprobado	build.txt
Jasmine y Karma	71 aprobadas, 0 fallidas	test-coverage.txt
Navegador real Chrome	15 grupos aprobados	browser/results.json y capturas

Cobertura frontend real

Métrica	Cubierto / total	Porcentaje
Statements	719 / 773	93.01%
Branches	532 / 604	88.07%
Functions	290 / 322	90.06%
Lines	233 / 237	98.31%

El punto de entrada de pruebas importa todos los módulos JS/JSX de aplicación, incluso si no tienen pruebas directas. Se excluyen las propias pruebas y main.jsx, que únicamente monta React e importa estilos. Los módulos no ejecutados permanecen en el denominador. La cobertura de líneas es alta, pero no equivale a cobertura completa: varias funciones comparten una línea en el código original.

Cobertura por módulo

Módulo	Sentencias	Ramas	Funciones	Líneas
App.jsx	100%	100%	100%	100%
ErrorBoundary.jsx	87.5%	100%	80%	85.71%
FormFields.jsx	100%	91.66%	100%	100%
ProductCard.jsx	100%	83.33%	100%	100%
ProductMedia.jsx	100%	82.14%	100%	100%
SiteLayout.jsx	82.41%	72.72%	74.35%	91.66%
ShopContext.jsx	97.31%	94.11%	96.15%	100%
catalog.js	100%	100%	100%	100%
AccountAdmin.jsx	95.23%	92.94%	93.33%	100%
CartCheckout.jsx	84.37%	90.9%	83.33%	100%
Home.jsx	85.29%	71.18%	84.21%	96.66%
InfoPages.jsx	100%	100%	100%	100%
ShopPages.jsx	97.36%	88.63%	94.11%	100%
commerce.js	100%	92.3%	100%	100%
storage.js	100%	100%	100%	100%

Mocks y aislamiento

jasmine.createSpy verifica las funciones recibidas por props. spyOn(window, fetch) entrega catálogos, pedidos, errores de red, respuestas HTTP inválidas y resultados del checkout sin red real. Se controla una promesa pendiente para comprobar el estado de carga. MemoryRouter aísla navegación. Cada especificación limpia localStorage, desmonta componentes y elimina el indicador de API. Los espías de confirmación permiten probar eliminación y restauración. Se simula falta de cuota de Storage y se valida el respaldo sin excepción.

La prueba de ErrorBoundary usa jasmine.spyOnGlobalErrorsAsync para capturar exclusivamente el error generado en ese caso, y comprueba que la interfaz ofrece recuperación. Las pruebas API ejecutan un servidor real en puerto 3107 con directorio temporal propio; no alteran el almacenamiento de desarrollo.

Hallazgos y correcciones

Línea base: 35/37 pruebas aprobadas; fallaban las dos esperas de API porque expect de Jasmine no lanza la excepción que waitFor necesita para reintentar. Se corrigió la espera, conservando assertions explícitas. Las nuevas pruebas detectaron que el catálogo restauraba productos eliminados al recargar, sobrescribía imágenes editadas y que ProductMedia alternaba indefinidamente entre respaldos fallidos. Esas conductas se corrigieron y quedaron cubiertas por regresiones.

Además se corrigieron: precio neto guardado en pedidos, mensaje específico de falta de stock, validación de campos numéricos vacíos en administración, rechazo de pedidos inexistentes en la ruta de éxito y tratamiento de errores al restaurar catálogo. El checkout del modo API ahora guarda pedidos y descuenta inventario en servidor.

Recorridos y diseño responsivo

Playwright recorre inicio, categorías, ofertas, detalle, carrito, checkout, rechazo, login, registro, contacto, nosotros, blog y 404 en 390×844, 768×1024, 1366×768, 1440×900 y 1920×1080. Comprueba hero de un viewport, tres indicadores, cambio de slide, navbar azul tras scroll, navegación móvil y ausencia de desbordamiento horizontal del documento.

El recorrido de compra filtra y busca, abre detalle, agrega un producto, modifica cantidades, verifica subtotal de $10.782 para dos unidades de Gran Master Hoja, recarga, reduce y elimina. Otro recorrido valida errores, rechaza sin crear pedido y aprueba conservando pedido y stock tras recargar. El CRUD crea, edita, elimina y recarga. Un contexto nuevo de navegador recibe la semilla. Se verifican imágenes con dimensiones naturales positivas y object-fit: contain.

Las evidencias están en testing/evidence/browser/. La ejecución no registró excepciones JavaScript ni respuestas HTTP fallidas. Las capturas corresponden a una ejecución real con movimiento reducido; el temporizador automático está presente en Home.jsx, pero su temporización completa no forma parte del recorrido E2E.

Ramas pendientes y alcance de la conclusión

Quedan sin ejecutar el 11,93% de ramas y el 9,94% de funciones instrumentadas. Incluyen alternativas de animación y temporizadores, eventos secundarios, recuperación por recarga y algunos respaldos o combinaciones opcionales. No se eliminó código ni se excluyeron componentes para elevar porcentajes. No se presenta la cobertura como exhaustiva al 100%. La prueba de navegador usa el modo local; el modo API se valida mediante mocks frontend y solicitudes HTTP reales en una suite separada.

Dependencias y límites

npm audit informó 11 avisos (7 moderados y 4 altos), registrados en dependency-audit.json. Las soluciones automáticas propuestas incluyen cambios mayores de React Router y retrocesos incompatibles de Karma. No se aplicaron cambios forzados. Las suites pasan en el entorno documentado; esto no certifica seguridad de producción ni compatibilidad con todos los navegadores.

Evaluación estricta del encargo

Indicador	Nivel justificado	Puntos
IE2.1.1 (10%)	100% técnico: arquitectura y flujos funcionales verificados; App, páginas y contexto.	10 / 10
IE2.1.2 (10%)	100% técnico: props, estado, Bootstrap y cinco tamaños comprobados.	10 / 10
IE2.2.1 (12%)	100%: 71 pruebas estructuradas y aprobadas en Jasmine/Karma.	12 / 12
IE2.3.1 (8%)	80%: entorno, mocks, análisis y cobertura alta; quedan ramas y comportamientos sin ejecutar.	6,4 / 8

Autoevaluación técnica: 38,4/40 puntos. No garantiza calificación docente.
Repositorio público: https://github.com/OscarMs1/Master-Martini
El código fuente y la documentación forman parte de esta entrega académica.

Matriz de casos ejecutados

Todos los casos siguientes finalizaron aprobados en la ejecución registrada. T corresponde a la suite original corregida y R a regresiones adicionales. Las assertions se ejecutan contra resultados de reglas o DOM reales.

Caso y comportamiento	Resultado
T01 — ProductCard renderiza nombre y formato usando props	Aprobado
T02 — ProductGrid presenta una tarjeta por cada producto recibido	Aprobado
T03 — Filtro por categoría devuelve exclusivamente elementos correspondientes	Aprobado
T04 — Catálogo muestra mensaje condicional si la búsqueda no tiene resultados	Aprobado
T05 — Evento de click ejecuta la función onAdd recibida por props	Aprobado
T06 — addCart respeta el stock y aumenta la cantidad sin duplicar filas	Aprobado
T07 — DOM carrito: aumentar cantidad actualiza el subtotal	Aprobado
T08 — DOM carrito: eliminar artículo activa estado vacío	Aprobado
T09 — Validación RUN usa dígito verificador y rechaza casos inválidos	Aprobado
T10 — Formulario checkout rechaza datos inválidos antes de finalizar	Aprobado
T11 — CRUD crear, modificar y eliminar devuelve listas inmutables	Aprobado
T12 — Validaciones impiden precio/stock negativos y checkout sin dirección	Aprobado
T13 — Slider de inicio: punto 2 cambia el título activo	Aprobado
T14 — Filtros React de portada cambian tarjetas sin recargar	Aprobado
T15 — Newsletter de muestra confirma suscripción	Aprobado
T16 — Campo accesible expone error y aria-invalid	Aprobado
T17 — Campo correcto no presenta error	Aprobado
T18 — Imagen de producto usa respaldo local cuando falla la URL remota	Aprobado
T19 — Producto agotado deshabilita botón añadir	Aprobado
T20 — La navbar muestra enlace de inicio y lista de categorías	Aprobado
T21 — Navbar móvil abre y cierra menú con estado React	Aprobado
T22 — El catálogo filtra por nombre desde un input controlado	Aprobado
T23 — Detalle de SKU inexistente entrega mensaje de error	Aprobado
T24 — ProtectedAdmin rechaza usuarios anónimos y redirige a login	Aprobado
T25 — Inicio de sesión inválido informa error de credenciales	Aprobado
T26 — Context API: clic modifica cartCount y persiste en almacenamiento	Aprobado

Caso y comportamiento	Resultado
T27 — Context API no agrega más del stock existente	Aprobado
T28 — Carrito inválido o agotado no puede completar pedido	Aprobado
T29 — Resumen: entrega retiro reduce costo de envío	Aprobado
T30 — Checkout valida correo con un mensaje accesible	Aprobado
T31 — Checkout tiene campos dependientes de región y comuna	Aprobado
T32 — Header carrito expone cantidad agregada	Aprobado
T33 — Botón secundario de portada dice Contáctanos en español	Aprobado
T34 — checkout válido registra compra simulada exitosa	Aprobado
T35 — pago rechazado no crea pedido ni cambia el inventario	Aprobado
T36 — Mock de fetch: carga datos simulados desde API sin red real	Aprobado
T37 — Mock de error API: presenta advertencia sin bloquear la app	Aprobado
R01 CRUD administrativo DOM conserva edición y eliminación después de remontar	Aprobado
R02 administración valida entradas vacías y SKU duplicado sin alterar catálogo	Aprobado
R03 pestañas de pedidos y usuarios y reinicio del catálogo	Aprobado
R04 vendedor consulta y edita pero no tiene botón eliminar	Aprobado
R05 registro inválido expone errores y no crea sesión	Aprobado
R06 registro válido conserva perfil sin contraseña y precarga checkout	Aprobado
R07 login administrador navega al panel y permite cerrar sesión	Aprobado
R08 cuenta muestra solamente pedidos del usuario y protege acceso anónimo	Aprobado
R09 formulario contacto valida y confirma envío simulado	Aprobado
R10 detalle aumenta y reduce cantidad y añade producto al carrito	Aprobado
R11 catálogo filtra categoría y ofertas y ordena por precio y nombre	Aprobado
R12 imágenes agotando ambos respaldos muestran estado accesible	Aprobado
R13 contexto respeta catálogo vacío guardado y carrito corrupto	Aprobado
R14 JSON roto y acceso denegado a storage tienen respaldo	Aprobado
R15 checkout guarda precio neto, total, stock y limpia carrito	Aprobado
R16 contexto bloquea datos inválidos, stock insuficiente y rechazo	Aprobado
R17 checkout muestra la causa de stock inválido y ruta falsa no confirma pedido	Aprobado

Caso y comportamiento	Resultado
R18 carrito se vacía mediante botón y checkout vacío ofrece catálogo	Aprobado
R19 límite de stock y producto agotado generan mensajes sin insertar filas	Aprobado
R20 CRUD contexto rechaza datos y edición inexistente y conserva imagen editada	Aprobado
R21 API carga catálogo y pedidos y expone estado de carga	Aprobado
R22 API rechaza datos malformados sin reemplazar catálogo válido	Aprobado
R23 API CRUD usa métodos correctos y mantiene estado si el servidor falla	Aprobado
R24 error HTTP con cuerpo no JSON se informa y bloquea checkout	Aprobado
R25 checkout API incorpora respuesta del servidor en pedidos y stock	Aprobado
R26 navegación App completa conserva SPA y llega al 404	Aprobado
R27 búsqueda navbar navega y Escape cierra panel	Aprobado
R28 portada cambia tercer slide y muestra diálogo de historia	Aprobado
R29 oferta del mes agrega el producto mostrado y filtros vacíos se explican	Aprobado
R30 blog navega al artículo y muestra error para slug inexistente	Aprobado
R31 página Nosotros expone alcance educativo y enlace al catálogo	Aprobado
R32 ErrorBoundary ofrece recuperación ante error de render	Aprobado
R33 reglas de carrito rechazan duplicados y entradas fuera de rango	Aprobado
R34 validación cubre precios, descuentos, correo y entrega inválidos	Aprobado

Reproducción y archivos de evidencia

Ejecutar npm install; npm run check:syntax; npm run check:logic; npm run test:node; npm run test:assets; npm run build; npm run test:coverage; npm run test:api. Para navegador, mantener npm run dev en otra terminal y ejecutar npm run test:e2e. testing/evidence/validation.json conserva los códigos de salida. testing/coverage/html/index.html muestra el detalle por línea; coverage-final.json conserva contadores completos y lcov.info permite integración con herramientas externas.